Quarto basics · reproducible documents

Day 4 - Thursday, May 21, 2026

Today: Thursday May 21

Before class

- **Perusall Assignment**: DC §4.1–4.7
  - (Optional) additional Quarto reading: *R for Data Science* Ch. 28 (Click here).

Plan for today

- Why reproducibility matters
- Anatomy of a `.qmd` file
- Code chunks and chunk options
- Rendering and the write/render cycle
- In-class activity

Helpful links

- Syllabus · AI policy · Schedule · *R4DS* Ch. 28

Quick recap from yesterday

Yesterday we covered:

- Reading R code: functions, arguments, data tables, variables, constants
- Calling functions with positional vs. named arguments
- Installing and loading **packages**
- The **tidyverse** as a coherent collection
- Building histograms with `ggplot2`

Today we'll learn how to wrap your code *and* your prose into a single reproducible document.

The problem with copy-paste

Imagine you've just finished a homework assignment in R. You:

1. Run your code in the Console.
2. Copy each plot into a Word document.
3. Type up your interpretation.
4. Submit.

Then your professor says: *"oh, please redo this with the updated data."*

> ⚠ Uh oh
>
> Now you have to re-do everything by hand. And to make things worse, every manual step is a place where mistakes happen.

Reproducibility

> 💡 A **reproducible** document is one where the report and the code that made it are the *same file*.

When the data changes, you just **re-render**. Every number, every plot, every table updates automatically.

This is what Quarto is for.

Quarto

What is Quarto?

Quarto is a system for writing documents that mix:

- **Prose** (regular text, headings, lists, links);
- **Code** (R, Python, Julia);
- **Output** (plots, tables, numbers from the code).

Quarto turns the whole thing into a polished **HTML page**, **PDF**, **Word doc**, **slide deck**, or even a website.

It's what I used to make the slides you're looking at right now.

Quarto vs. R Markdown

You might hear about **R Markdown** (`.Rmd` files) - it's Quarto's older sibling.

- R Markdown came first; it works only with R.
- Quarto is newer; works with R, Python, Julia, and more.
- Both use the same underlying ideas.

In this class we'll use **Quarto** (`.qmd` files) exclusively.

> **Note**
>
> If you see `.Rmd` examples online (including in the *Data Computing* book), they translate almost directly to `.qmd`. Don't worry too much about the difference.

What's in a `.qmd` file?

A Quarto document is a plain text file with three kinds of content:

1. A **YAML header** - settings, surrounded by `---`
2. **Code chunks** - R code, surrounded by ```` ```{r} ```` and ```` ``` ````
3. **Markdown text** - your prose, with formatting like `# headings` and `**bold**`

You can open a `.qmd` file in any text editor.

A complete example

````
---
title: "Penguin sizes"
author: "Charles Costanzo"
format: html
---

# Introduction

We have data on 344 penguins
from three species.

```r
library(palmerpenguins)
library(ggplot2)
````
````

````
ggplot(penguins, aes(x = body_mass_g)) +
  geom_histogram(bins = 30,
                 color = "black",
                 fill = rgb(0, 206/255, 209/255)) +
  theme_classic() +
  labs(title = "Distribution of Body Mass (grams)",
       x = "Body Mass (grams)",
       y = "Count of Penguins")
```

The distribution looks roughly **bimodal**, with two peaks.
````

Rendered output

- Three pieces: YAML header, prose, code chunk. The output is a polished HTML page (as promised).

The YAML header

What's a YAML header?

The block at the very top, between the two `---` lines:

```
---
title: "Penguin sizes"
author: "Charles Costanzo"
date: "2026-05-21"
format: html
---
```

This tells Quarto things like:

- What's the title?
- Who's the author?
- What format should I render to?

> Fun Fact
>
> **YAML** originally stood for "Yet Another Markup Language." However, unlike HTML or XML, which are designed to mark up and format text documents, YAML was designed
```

strictly for data serialization: storing data, lists, and configuration settings in a way humans can easily read. To clarify its actual purpose, the creators later changed it to "YAML Ain't Markup Language." This makes it a "recursive acronym" (where the Y stands for YAML itself).

YAML syntax rules

YAML is **picky** about formatting:

```yaml
---
title: "My Report"
format:
  html:
    toc: true
    embed-resources: true
---
```

⚠ YAML gotchas

- **Indentation matters.** Use spaces, not tabs. Two spaces per level is standard.
- **Colons need a space after them.** `title: "Hi"`, **not** `title:"Hi"`.
- **Strings with special characters need quotes.** Use `"My: Report"`, **not** `My: Report`.

If your YAML breaks, your document won't render. Most error messages will point to the line number.

Common format options

```yaml
format: html
```

```yaml
format: pdf
```

```yaml
format: docx
```

You can also render to **multiple** formats at once:

```yaml
format:
  html:
    toc: true
  pdf:
    fig-pos: "H"
```

> **Note**
>
> PDF rendering needs **LaTeX** installed on your machine. RStudio will prompt you to install it the first time you try.

Code chunks

What's a code chunk?

A **code chunk** is a block of R code embedded in your document. It looks like this:

````
```{r}
2 + 2
```
````

When you render the document, Quarto:

1. Finds each chunk.
2. Runs the R code inside.
3. Inserts both the code *and* its output into the final document.

So your readers see your work *and* its results, side-by-side, always in sync.

Three ways to insert a chunk

1. **Keyboard shortcut** (recommended):
   - Mac: `Cmd + Option + I`
   - Windows/Linux: `Ctrl + Alt + I`
2. **Toolbar button**: the green `+C` icon at the top of the editor.
3. **Type it yourself**: ` ```{r} ` on one line, your code, ` ``` ` to close.

> **Tip**
>
> Learn the keyboard shortcut. You'll use it a lot.

Running chunks interactively

You don't have to render the whole document every time. While editing, you can run chunks one at a time:

Table 1: Useful chunk-running shortcuts.

Action	Shortcut
Run the current line	`Cmd/Ctrl + Enter`
Run the whole chunk	`Cmd/Ctrl + Shift + Enter`
Run all chunks above	(button at top of chunk)

This is exactly like working in the Console, but your code stays saved in the document.

Chunk options

You can control what each chunk does by adding options at the top, prefixed with `#|`:

````
```{r}
#| label: my-plot
#| echo: false
#| fig-width: 6

ggplot(penguins, aes(x = body_mass_g)) +
  geom_histogram()
```
````

Common options:

Table 2: Common chunk options.

Option	What it does
`echo: false`	Hide the code, show the output
`eval: false`	Show the code, don't run it
`include: false`	Run the code, hide everything

Option	What it does
`message: false`	Hide messages (e.g., from `library()`)
`warning: false`	Hide warnings

When to use which option

> Quick decision guide
>
> - Writing a **report for a non-coder?**: `echo` → `false` everywhere.
> - **Loading libraries** in your setup chunk?: `include` → `false` to hide the noise.
> - **Showing example code** you don't want to run?: `eval` → `false`.
> - **Getting annoying red messages** when loading the tidyverse?: `message` → `false`.

You can set options chunk-by-chunk, or globally in the YAML header (more on that in a bit).

The setup chunk

A common pattern: have one chunk at the **top** of your document that loads packages and sets defaults.

````
```{r}
#| label: setup
#| include: false

library(tidyverse)
library(palmerpenguins)
```
````

- `label: setup` is a **special name** - Quarto runs this chunk first, automatically.
- `include: false` hides it from the rendered output (your reader doesn't need to see `library(tidyverse)`).

Markdown

Markdown basics

Outside code chunks, your prose is written in **Markdown** - a simple syntax for formatting plain text.

Table 3: Common Markdown syntax.

You type	You get
`*italic*`	*italic*
`**bold**`	**bold**
`` `code` ``	`code`
`# Heading 1`	(big heading)
`## Heading 2`	(smaller heading)
`- item`	bulleted list item
`1. item`	numbered list item
`[link text](https://example.com)`	link text

More Markdown

```
> This is a block quote.
> It gets a vertical bar on the left.
```

```
| Column 1 | Column 2 |
|----------|----------|
| Apple    | Red      |
| Banana   | Yellow   |
```

> Shortcut
>
> In RStudio: `Help > Markdown Quick Reference` opens a cheat sheet you can keep open while you write.

Visual editor vs. source editor

RStudio has **two ways** to edit a Quarto document:

- **Source** - you see the raw Markdown and YAML. Like editing code.
- **Visual** - looks more like Google Docs or Word. Buttons for bold, italic, headings, etc.

There's a toggle in the top-left of the editor pane.

> **Note**
>
> Both editors save the **same** plain-text `.qmd` file underneath. You can switch back and forth freely.

I recommend starting in **Visual** mode if you're new to Markdown - you'll learn the syntax by seeing it appear.

Rendering

How rendering works

When you click **Render**, Quarto:

1. Reads your `.qmd` file.
2. Sends it to `knitr`, which runs each code chunk in order.
3. Sends the result to `pandoc`, which converts everything to your target format.
4. Saves the final document next to your `.qmd` file.

> **Fresh session**
>
> Quarto runs your code in a **brand new R session** every render. It does **not** know about objects you made in the Console.
> If your code relies on something you typed in the Console, rendering will fail. Everything the document needs must be in the document.

The render button

In RStudio:

- Click **Render** in the editor toolbar, or
- Press `Cmd/Ctrl + Shift + K`

The output opens automatically in the Viewer pane (or your browser).

The write / render cycle

> Render early, render often.

Don't write 200 lines and then try to render. Write a little, render, check it works, write a little more.

If you make a mistake in chunk #47, it's much easier to find when you only added chunks #45–47 since your last successful render.

Activity

Activity: your first Quarto report

You'll build a short Quarto report from scratch. Take 25 minutes for this activity.

- Activity Canvas Link

To turn in: Save your `.qmd` file and the rendered HTML. We'll check at the end of class.

Activity debrief

Common stumbling blocks:

- **YAML indentation** - extra space or missing space at the start of a line breaks everything.
- **Forgot to close a chunk** - every ```` ```{r} ```` needs a matching ```` ``` ```` on its own line.
- **Object not found** during render - you typed it in the Console but not in the document. The render uses a fresh session.
- **Package not installed** - `Error: there is no package called 'palmerpenguins'`. Install it in the Console.
- **Plot doesn't appear** - usually a missing `+` between `ggplot()` layers.

Wrap-up: what we covered today

- Recognize a `.qmd` file as a mix of YAML, prose, and code chunks.
- Write a YAML header with title, author, and format.
- Insert and run code chunks.
- Hide code, output, messages, or warnings with chunk options.
- Use basic Markdown for headings, lists, bold, italic, and links.
- Render a document to HTML.

Wrap-up & looking ahead

Tomorrow: Quiz 1, then the pipe operator and chaining syntax.

Upcoming Deadlines

- **Fri 5/22, before class** - DC §3.6
- **Fri 5/22, in class** - Quiz 1
  - Quiz 1 Practice
- **Fri 5/22, 11:59 p.m.** - HW 1: Basic R & Plots

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

Syllabus · AI policy · Schedule · *R4DS* Ch. 28